Deriving Data Structures and Functions from the Specification

> *“How do I derive the data structures and functions from the specification, instead of just copying code?”*

Let's separate two things:

1. What the IP-00 specification explicitly tells us.
2. What we, as designers, must implement to make the specified bus work.

1. Where did struct Region come from?

First, look at your IP-00 specification, Section 3, *Bus — the interface you call*, on page 2. It declares these methods:

```
void register_slave(
    const std::string& name,
    uint32_t base,
    uint32_t size,
    IBusSlave* slave
);

bool transact(uint32_t addr, uint32_t& data, bool is_write);
bool write(uint32_t addr, uint32_t data);
bool read(uint32_t addr, uint32_t& data);
```

The specification tells us that the bus must register slave address windows and route transactions to the correct slave. It also explains that address decoding uses a linear scan of registered regions.

But it does not prescribe a `struct Region` in the excerpt we've examined. That was an implementation choice in my temporary teaching bus.

Let's derive why it is useful. Imagine you register three IPs:

IP	Base address	Size
RAM	0x30000000	4096 bytes
DMA	0x20000200	24 bytes
Flash Interface	0x20000400	20 bytes

For every IP, the bus needs to remember four things:

- Its name.
- Its starting address.
- The size of its address window.
- A reference to the object that handles accesses to that window.

That gives us this data structure:

```
struct Region {
    std::string name;
    uint32_t base;
    uint32_t size;
    IBusSlave* slave;
};
```

Think of one `Region` object as a record in the bus's address map.

2. Why do we need std::vector<Region>?

Once we have defined a record for one address window, the bus needs somewhere to store multiple records. For example:

```
private:
    std::vector<Region> regions;
```

The reasoning is:

1. The SoC contains multiple IPs.
2. Each IP registers its address window.
3. The number of registered IPs may grow as we add more IPs.
4. Therefore, the bus needs a collection of address-window records.

A vector is a convenient choice because we can add entries with `push_back()` and iterate over them when decoding an address.

The specification requires the bus to register and decode regions; the particular choice of `std::vector` is our implementation decision, not a mandated specification detail.

3. How do we derive each bus function?

Let's work through the four methods individually, using the same question each time: **What responsibility does the specification assign to this function?**

A. register_slave() — remember an IP

The specification gives us this interface:

```
void register_slave(
    const std::string& name,
    uint32_t base,
    uint32_t size,
    IBusSlave* slave
);
```

Ask yourself: when RAM calls this function from its constructor, what information must the bus remember so it can find RAM later? The answer is precisely the four fields in `Region`. That leads to the implementation:

```
void register_slave(const std::string& name,
                    uint32_t base,
                    uint32_t size,
                    IBusSlave* slave) {
    regions.push_back({name, base, size, slave});
}
```

Here, `regions.push_back(...)` creates a new entry containing the supplied information. The flow is:

```
RAM constructor calls register_slave(...)
        |
        v
Bus stores the address window and RAM pointer
        |
        v
Bus can locate RAM during a later transaction
```

Important: registering RAM doesn't perform a memory read or write. It only establishes the mapping.

B. transact() — route a transaction

The specification says that `transact()` returns false when an address is unmapped, and an unmapped read sets data to zero. It also specifies that the real bus uses a mutex and permits one outstanding transaction at a time.

Let's derive the algorithm:

1. Receive a full system address.
2. Search the registered regions.
3. Find the region containing the address.
4. Convert the full address into an offset relative to that region.
5. Call that slave's `bus_access()`.
6. Return true if the transaction reached a mapped region.
7. If no region matches, return false and set read data to zero.

The central address test is:

```
addr >= region.base &&
uint64_t(addr) < uint64_t(region.base) + region.size
```

Why use a 64-bit expression for the upper bound? It prevents the addition of a 32-bit base and size from wrapping around near the maximum 32-bit address.

The relative address is then calculated as:

```
uint32_t offset = addr - region.base;
```

For example, if the RAM base is 0x30000000 and the transaction address is 0x30000024, the offset is 0x24.

The routing operation is:

```
region.slave->bus_access(offset, data, is_write);
```

This is **polymorphism** in C++: the bus knows only about the `IBusSlave` interface, so it can call RAM, DMA, or another IP through the same interface.

Why is the real transact() blocking?

In your specification, the real bus uses an `sc_mutex` to protect transactions. Acquiring the mutex can block until another transaction finishes. That is why the specification restricts bus calls to processes in which blocking is legal, such as `SC_THREAD`. (Source: IP00_Bus_and_Slave_Interface_Spec.pdf)

The simplified bus we wrote earlier doesn't implement that mutex or the corresponding blocking behavior. Its `transact()` is a functional routing model only. We must not confuse it with the complete supplied framework.

C. write() — provide a convenient write operation

The specification declares:

```
bool write(uint32_t addr, uint32_t data);
```

Think about how a CPU testbench should write a register. It shouldn't have to manually prepare every argument to `transact()`. We can implement `write()` as a wrapper:

```
bool write(uint32_t addr, uint32_t data) {
    return transact(addr, data, true);
}
```

Why `true`? Because `is_write` must be true for a write transaction.

Notice that `data` is passed by value to `write()`, but `transact()` accepts it by reference. That is fine: `transact()` receives a local copy of the data, which is all a write needs.

D. read() — provide a convenient read operation

The specification declares:

```
bool read(uint32_t addr, uint32_t& data);
```

Here, `data` is passed by reference because the caller needs to receive the result. The implementation is:

```
bool read(uint32_t addr, uint32_t& data) {
    return transact(addr, data, false);
}
```

Why `false`? Because this is a read transaction. For example:

```
uint32_t value = 0;

if (bus.read(0x30000024, value)) {
    std::cout << std::hex << value << '\n';
}
```

The bus routes the read to RAM, RAM assigns the returned word to `value`, and the caller can then use it.

4. Put all four functions together

Here's the complete relationship:

```
register_slave()      -> Builds the address map

write()               -> Calls transact(..., true)
read()                -> Calls transact(..., false)

transact()            -> Finds the region, computes the offset,
                         and dispatches the request.

IBusSlave::bus_access() -> The selected IP performs the read or write.
```

5. Your most important design distinction

You asked whether we should declare these functions and also implement them. The answer depends on which part of the project you're working on:

Situation	What should you do?
Building the bus from scratch for this temporary learning exercise	Declare and implement all four methods.
Implementing IP-01 in the actual project	Use the provided Bus and IBusSlave definitions; implement RAM's constructor and bus_access().
The real project has declarations but no implementations	Locate the framework source and build instructions before deciding whether anything is missing. Your specification says IP-00 is provided and must not be modified.

The temporary `Region` structure and bus methods were my way of creating a small environment for learning while you only had the specifications. They are not additional requirements for your RAM IP.